

Technical Case Studies: Three Defect Classes in Browser-Driven Agent Automation

Short, blog-post-length writeups. Each documents a real defect found and fixed in this repository, the general defect *class* it belongs to (so the writeup is useful to someone who has never seen this codebase), and what made it non-obvious. None of these are claimed as novel defect classes — each is a named, recognized category in software engineering; what's documented here is a concrete instance and its root-cause analysis.

Case Study 1: Unescaped Interpolation into a Query Language (XPath/JS Injection)

Defect class: Improper Neutralization of Special Elements used in a Query Language (the general pattern underlying SQL injection, and here applied to XPath 1.0 and to a dynamically constructed JavaScript expression) — CWE-943 generalizes this; the XPath-specific form is sometimes separately tracked as CWE-643.

Instance: `DefaultActionWatchdog.on_ScrollToTextEvent()` took agent-supplied (ultimately LLM-supplied) text and spliced it directly into three XPath query strings via f-string interpolation (`f'//*[contains(text(), "{event.text}")]'`) and, in a fallback path, into a JavaScript string literal inside a `Runtime.evaluate()` expression. Any target text containing a double-quote character terminated the enclosing string literal early in both the XPath and the JS forms, producing a syntactically different query than intended.

What made this non-obvious: the resulting behavior was not a crash. The XPath engine (Chrome DevTools Protocol's `DOM.performSearch()`) is permissive by design — a malformed XPath expression degrades silently to a plain-text substring search on the *literal* (broken) query string rather than raising an error, so the broken query simply matches nothing and the function reports "text not found." A naive read of the code predicts an exception (caught by a broad `except: continue`); the actual failure mode has no exception at all, at any layer, until the final user-facing "not found" error two call-frames later. **The reproduction that mattered was empirical** — instrumenting the actual CDP call and confirming zero exceptions were thrown anywhere in the chain — not a static read of the code, which would have predicted the wrong mechanism.

Fix shape: the JavaScript path takes the standard remediation — `json.dumps()` before interpolation, which is safe here because JSON string-literal escaping and JavaScript string-literal escaping are compatible for this character set. XPath 1.0 has no analogous escape sequence for embedded quote characters inside a string literal, so the fix implements the standard workaround for that specific limitation: split the value on `"`, wrap each segment as a `"..."` literal, and reassemble with XPath's `concat()` function, falling back to a `'...'`-delimited literal when the value contains only one quote character and

can use the other delimiter directly.

Case Study 2: Remote-Object Lifecycle Leak Under an Object-Group Abstraction

Defect class: Resource leak via incomplete cleanup on a non-happy path, specifically in a client managing remote handles owned by a separate process (CWE-401, "Missing Release of Memory after Effective Lifetime" — here the "memory" is a CDP inspector-backend object reference, not process-local heap memory, so the leak lives in the browser process, not the Python process).

Instance: `DomService._get_all_trees()`'s JS click-listener detection resolves up to 100 individual element handles via `Runtime.getProperties()` against an array object obtained from `Runtime.evaluate()`. The code released only the parent array's handle (`Runtime.releaseObject` on the array's own `objectId`); every child handle resolved from `getProperties()` was left live in the renderer's V8 inspector backend indefinitely — for the remaining lifetime of that CDP session, which for a long-running agent can be hours.

What made this non-obvious: nothing about the leak is visible from Python's perspective. There is no Python-side reference, no garbage collector pressure, no exception. The resource is entirely on the other side of the CDP boundary. **Verifying it required treating it as a distributed-systems problem, not a language-runtime one:** capture the real object IDs the browser actually returned during a genuine `Runtime.getProperties()` call, then independently attempt to release one a second time and observe CDP's own error ("Could not find object with given id") as the ground truth for "was this handle already reclaimed." On the buggy code, the second release always succeeded (proving the handle was still live); on the fixed code, it always failed with that exact error (proving it had already been reclaimed).

Fix shape: CDP's `Runtime` domain provides exactly the abstraction this needed — an `objectGroup` string tag, settable at `Runtime.evaluate()` time, which every property handle later resolved from that object inherits, plus a single `Runtime.releaseObjectGroup()` call that frees the whole group regardless of how many handles it ended up containing. This is the standard pattern the CDP `Runtime` domain exists to support; the defect was that the original code released a plain object handle instead of using the tagged-group mechanism.

Case Study 3: State Corruption via Variable Reuse Across an Unrelated Control-Flow Branch

Defect class: this doesn't have a single crisp CWE — it's closest to CWE-664 ("Improper Control of a Resource Through its Lifetime") combined with a classic variable-shadowing/reuse bug, made security-relevant here because the shared variable also happened to gate a stateful side effect (changing which browser tab has "focus" in this system's own session model) rather than being purely a read.

Instance: `CrashWatchdog._check_browser_health()` bound a local variable (`cdp_session`) to the CDP session for the agent's current focus target, intending to use that same variable for a "quick ping" responsiveness check at the end of the function. In between, an unrelated per-tab cleanup loop (redirecting any stray `chrome://new-tab-page/` tabs to `about:blank`) *reassigned that same variable* inside its loop body for each tab it processed. Because the session-lookup helper used to get a tab's CDP session defaults to also switching the system's notion of "focused tab" as a side effect, this reassignment did not just leave a stale reference in a local variable — it actively moved the real focus state onto whichever background tab the cleanup loop happened to touch last, and the subsequent "quick ping" then checked *that* tab's responsiveness instead of the one the function's own log messages claimed it was checking.

What made this non-obvious: every log line printed during execution was internally self-consistent and pointed at the (by-then-relocated) focus target — reading the logs in isolation gives no signal that anything is wrong; the log lines are correct given the code's actual (bugged) behavior, they simply don't reflect the caller's original intent 30 lines earlier. Confirming the defect required constructing the specific interleaving that exposes it (a genuinely hung renderer on the real focus tab, *plus* a stray background tab present at the same time) and checking the health-check's verdict against ground truth established independently (`pytest.raises` on a direct evaluation against the known- hung tab, proving the hang is real, before ever invoking the buggy function). A control case with the identical hang and no stray tab was required to rule out "the health check is broken in general" as a simpler, wrong explanation.

Fix shape: bind the cleanup loop's session lookup to its own, differently-named local variable, and pass `focus=False` to the session-lookup call inside that loop so the cleanup's own housekeeping never has the side effect of moving the system's focus state — the narrowest fix that addresses the actual coupling (an unrelated helper's default side effect leaking into a variable-reuse bug) rather than only the symptom (the variable name collision).

Scope note, stated honestly: this specific watchdog is not currently instantiated anywhere in the codebase (its construction is commented out), so as of this writing the defect has zero effect on any shipped configuration. It is documented here because the underlying pattern — a shared-side-effect helper's default behavior leaking through a reused local variable — is a real, generally applicable failure mode, not because this specific instance is currently exploitable or user-visible.